Functions- call by value, call by reference, nesting of functions, recursive functions

C – Functions

```
void myFunction(char name[]) {
  printf("Hello %s\n", name);
}

int main() {
  myFunction("ABC");
  myFunction("XYZ");
  myFunction("ijk");
  return 0;
}

// Hello ABC
// Hello XYZ
// Hello ijk
```

When a **parameter** is passed to the function, it is called an **argument**. So, from the example : `name` is a **parameter**, while `ABC`, `XYZ` and `ijk` are **arguments**.

Note
When you are working with multiple parameters, the function call must have the same number of arguments as there are parameters, and the arguments must be passed in the same order.

Pass Arrays as Function Parameters

```
void myFunction(int myNumbers[5])
{
  for (int i = 0; i < 5; i++)
 {
    printf("%d\n", myNumbers[i]);
  }
}

int main()
{
  int myNumbers[5] = {10, 20, 30, 40, 50};
  myFunction(myNumbers);
  return 0;
}
```

Note that when you call the function, you only need to use the name of the array when passing it as an argument `myFunction(myNumbers)`. However, the full declaration of the array is needed in the function parameter (`int myNumbers[5]`).

Pass by value / Call by value

We can pass arguments to the C function in two ways:

- Pass by Value
- Pass by Reference

1. Pass by Value

Parameter passing in this method copies values from actual parameters into formal function parameters. As a result, any changes made inside the functions do not reflect in the caller's parameters.

Example:

```
// C program to show use of call by value
#include <stdio.h>
void swap(int var1, int var2)      // formal arguments
{
    int temp = var1;
    var1 = var2;
     var2 = temp;
}
int main()
{
   int var1 = 3, var2 = 2;
   printf("Before swap Value of var1 and var2 is: %d, %d\n", var1, var2);
   swap(var1, var2);     // Actual arguments
   printf("After swap Value of var1 and var2 is: %d, %d", var1, var2);
   return 0;
}
```

Output

```
Before swap Value of var1 and var2 is: 3, 2
After swap Value of var1 and var2 is: 3, 2
```

Pass by reference / Call by reference

2. Pass by Reference

The caller's actual parameters and the function's actual parameters refer to the same locations, so any changes made inside the function are reflected in the caller's actual parameters.

```
// C program to show use of  call by Reference
#include <stdio.h>
void swap(int *var1, int *var2)
{
    int temp = *var1;
    *var1 = *var2;
    *var2 = temp;
 }
int main()
{
   int var1 = 3, var2 = 2;
   printf("Before swap Value of var1 and var2 is: %d, %d\n", var1, var2);
   swap(&var1, &var2);
   printf("After swap Value of var1 and var2 is: %d, %d", var1, var2);
   return 0;
}
```

Output

Before swap Value of var1 and var2 is: 3, 2
After swap Value of var1 and var2 is: 2, 3

C Variable Scope

In C, variables are only accessible inside the region they are created. This is called **scope**.

Local Scope

- A variable created inside a function belongs to the *local scope* of that function, and can only be used inside that function:
- A **local variable** cannot be used outside the function it belongs to.
- If you try to access it outside the function, an error occurs:

```
void myFunction()
{
   int x = 5; // Local variable that belongs to myFunction
   printf("%d", x);
}

int main()
{
  myFunction();
  //printf("%d", x);  // will create error
  return 0;
}
```

C Variable Scope

Global Scope

- A variable created outside of a function, is called a **global variable** and belongs to the *global scope*.
- Global variables are available from within any scope, global and local:
- A variable created outside of a function is global and can therefore be used by anyone:

```
#include<stdio.h>
int x = 5; // Global variable x

void myFunction()
{
  printf("%d", x); // We can use x here
}

int main()
{
  myFunction();
  printf("%d", x); // We can also use x here
  return 0;
}
```

Naming Variables

If you operate with the **same variable name inside and outside of a function, C will treat them as two separate variables**; One available in the global scope (outside the function) and one available in the local scope (inside the function):

- However, **you should avoid using the same variable name for both globally and locally variables as it can lead to errors and confusion.**
- Be careful with global variables, since they can be accessed and modified from any function:

```
#include<stdio.h>
int x = 5; // Global variable x

void myFunction()
{
  int x = 22; // Local variable with the same name as the global variable (x)
  printf("%d\n", x); // Refers to the local variable x
}

int main()
{
 myFunction();
 printf("%d\n", x); // Refers to the global variable x
 return 0;
}
```

Nested Functions in C

The term **nesting**, in programming context refers to enclosing a particular programming element inside another similar element. Just like nested loops, nested structures, etc., a nested function is a term used to **describe the use of one or more functions inside another function.**

Lexical Scoping

- In C language, defining a function inside another one is **not possible**. In short, nested functions are not supported in C. A function **may only be declared (not defined) within another function**.
- When a function is declared inside another function, it is called lexical scoping. Lexical scoping is not valid in C because the compiler cannot reach the correct memory location of inner function.
- **Nested Functions Have Limited Use**
- Nested function definitions cannot access local variables of surrounding blocks. They can access only global variables. In C, there are two nested scopes: local and global. So, nested functions have limited use.

Nested Functions in C

Trampolines for Nested Functions

- Nested functions are supported as an extension in "GNU C". GCC implements taking the address of a nested function using a technique called **trampolines**.
- A trampoline is a piece of code created at runtime when the address of a nested function is taken. It requires the function to be prefixed with the keyword **auto** in the declaration.

```
#include <stdio.h>
int main()
{
    auto int nested();
    nested();
    printf("In Main Function now\n");
    int nested()
    {
       printf("In the nested function now\n");
    }
   printf("End of the program");
}
```

Output

```
In the nested function now
In Main Function now
End of the program
```

Recursive Functions in C

Recursion is the process of a function calling itself repeatedly till the given condition is satisfied.

A function that calls itself directly or indirectly is called a recursive function and such kind of function calls are called recursive calls.

In C, recursion is used to solve complex problems by breaking them down into simpler sub-problems.

For example, factorial of a number, generating Fibonacci series, generating subsets, etc.

To add a range of numbers (sum of 10 natural numbers) together by breaking it down into the simple task of adding two numbers:

```
#include<stdio.h>
int sum(int k);
int main()
{
  int result = sum(10);
  printf("%d", result);
  return 0;
}
int sum(int k)
{
  if (k > 0) {
    return k + sum(k - 1);
  } else {
    return 0;
  }
}
```

```
10 + sum(9)
10 + ( 9 + sum(8) )
10 + ( 9 + ( 8 + sum(7) ) )
...
10 + 9 + 8 + 7 + 6 + 5 + 4 +3+2+1+sum(0)
```

Explanation

When the `sum()` function is called, it adds parameter `k` to the sum of all numbers smaller than `k` and returns the result. When k becomes 0, the function just returns 0. When running, the program follows these steps:

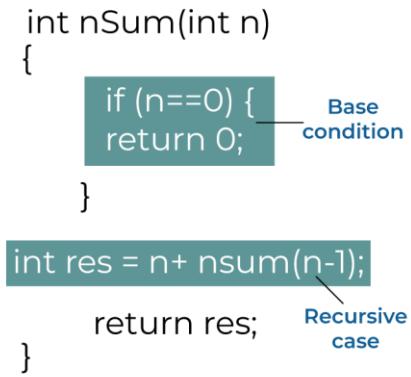
Since the function does not call itself when k is 0, the program stops there and returns the result.

Recursive Functions in C

Fundamentals of C Recursion

The fundamental of recursion consists of two objects which are essential for any recursive function. These are:

- Recursion Case
- Base Condition



1. Recursion Case

The recursion case if often represented mathematically is a **recurrence relation.** For the above case:

- F(N) = N + f(N - 1);

2. Base Condition

- The base condition specifies when the recursion is going to terminate. It is the condition that determines the exit point of the recursion.
- **Note:** It is important to define the base condition before the recursive case otherwise, the base condition may never encountered and recursion might continue till infinity.

```
if (n == 0)
    {
       return 0;
    }
```

Recursive Functions in C – Factorial of a number

```
#include <stdio.h>
int fact (int);
int main()
{
   int n,f;
   printf("Enter the number whose factorial you
                                want to calculate?");
   scanf("%d",&n);
   f = fact(n);
   printf("factorial = %d",f);
}
int fact(int n)
{
   if (n==0)
   {
      return 0;
   }
   else if ( n == 1)
   {
      return 1;
   }
   else
   {
      return n*fact(n-1);
   }
}
```

Recursive Functions in C – Fibonacii series

```
#include <stdio.h>
void printFibonacci(int n)
{
  int n1 = 0, n2 = 1, n3;
  if (n > 0)
  {
    n3 = n1 + n2;
    n1 = n2;
    n2 = n3;
    printf("%d ", n3);
    printFibonacci(n - 1);
  }
}
```

```
int main()
{
int n;
printf("Enter the number of elements: ");
scanf("%d", &n);

printf("Fibonacci Series: ");
printf("%d %d ", 0, 1); // printing 0 and 1
printFibonacci(n - 2); // n-2 because 2 numbers are already printed

return 0;
}
```

```
Enter the number of elements: 8
Fibonacci Series: 0 1 1 2 3 5 8 13
```

Recursive Functions in C

Applications of Recursion in C

Recursion is widely used to solve different kinds of problems from simple ones like printing linked lists to being extensively used in AI. Some of the common uses of recursion are:

- Tree-Graph Algorithms
- Mathematical Problems
- Divide and Conquer
- Dynamic Programming
- In Postfix to Infix Conversion
- Searching and Sorting Algorithms

Advantages of C Recursion

- Recursion can effectively reduce the length of the code.
- Some problems are easily solved by using recursion like the tower of Hanoi and tree traversals.
- Data structures like linked lists, trees, etc. are recursive by nature so recursive methods are easier to implement for these data structures.

Disadvantages of C Recursion

- Recursive functions make our program a bit slower due to function call overhead.
- Recursion functions always take extra space in the function call stack due to separate stack frames.
- Recursion methods are difficult to understand and implement.